

Maximal Directed k -Plex Mining

Akhlaque Ahmad*, Da Yan*, Jiao Han*, Jalal Khalil[†], Lyuheng Yuan*, Saugat Adhikari*, Yang Zhou[‡]
^{*}Indiana University Bloomington, USA; Emails: {akahmad, yanda, jiaohan, lyyuan, adhiksa}@iu.edu

[†]St. Cloud State University, USA; Email: jalal.khalil@stcloudstate.edu

[‡]Auburn University, USA; Email: yangzhou@auburn.edu

Abstract—Finding cohesive subgraphs in a large graph has many important applications, such as community detection and biological network analysis. Clique is often too strict since communities or biological modules rarely form as cliques for various reasons such as data noise. Therefore, k -plex is introduced as a popular clique relaxation, which allows each vertex to miss up to k neighbors (including itself). However, the existing k -plex definition and algorithms are only applicable to undirected graphs, while many real networks are directed.

In this paper, we generalize the concept of k -plex to directed graphs by proposing (k_1, k_2) -plex which has connectivity requirements in both inbound and outbound directions of each vertex in a subgraph. An efficient recursive algorithm is proposed to find maximal (k_1, k_2) -plexes which integrates many effective techniques for search space pruning. The algorithm is parallelized with effective load balancing, and we demonstrate that it can scale up effectively with the number of CPU cores. Our code has been released at <https://github.com/akhlaqueak/k1k2-plex>.

I. INTRODUCTION

The concept of k -plex was proposed in [1] as a relaxation of clique. Specifically, given a simple undirected graph G , a k -plex is a subgraph of G , denoted by $g = (V_g, E_g)$, where each vertex connects to at least $(|V_g| - k)$ other vertices in g . In other words, any vertex $v \in V_g$ can miss at most k edges to the vertices in V_g (including v itself as a self-loop). It has found extensive applications in the analysis of social networks [1], especially in community detection [2], [3].

However, k -plex is defined for undirected graphs, while many real networks are directed such as gene regulatory networks and Twitter follower networks. So, it is important to define and mine meaningful directed k -plex counterpart. We identify the importance of having degree requirements in both inbound and outbound directions of each vertex, which leads to the concept of (k_1, k_2) -plex, denoted by g , where each vertex connects to at least $(|V_g| - k_1)$ other vertices in g , and is meanwhile pointed to by at least $(|V_g| - k_2)$ other vertices in g .

To illustrate, Figure 1(a) shows a 3-plex where each vertex disconnects at most 3 vertices (including itself). If we make the edges directed as in Figure 1(b) and add another vertex f that points to every other vertex, we can see that each vertex therein still disconnects at most 3 vertices (outbound). However, we cannot regard f as a member of the dense subgraph. Consider, for example, a group of Twitter users that frequently interact with each other; then the fact that a new user f follows and retweets many users in that group does not make f a member, unless a significant number of users in the group also pay attention to f . In contrast, the graph g in Figure 1(c) forms

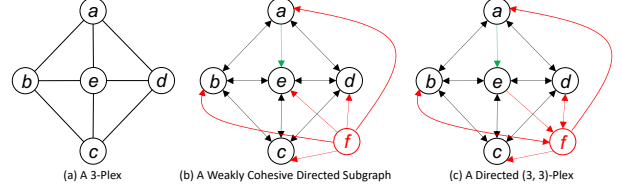


Fig. 1. Motivations for Defining (k_1, k_2) -Plexes

a convincing dense group since every vertex points to and is also pointed to by at least $3 = (|V_g| - 3)$ vertices, making it a $(3, 3)$ -plex (i.e., $k_1 = k_2 = 3$). Besides this example, (k_1, k_2) -plexes can also be used in many other applications in areas including bioinformatics (e.g., finding co-regulated genes [4]) and cybersecurity (e.g., botnet detection [5]).

However, mining k -plexes is NP-hard [6], [7], so existing algorithms rely on branch-and-bound search which runs in exponential time in the worst case. Many recent works have studied the branch-and-bound algorithms for mining maximal k -plexes [8], [3], [9], [10], [11] and finding a maximum k -plex [12], [13], [14], [15], [16], [17], [18], [19], with various techniques proposed to prune the search space. Our (k_1, k_2) -plex definition generalizes k -plex, and in fact, if we treat an undirected graph G as bidirected and mine (k, k) -plex on it, this special case is equivalent to mining k -plex on G . Therefore, mining maximal (k_1, k_2) -plex is also NP-hard.

In this paper, we design an efficient algorithm to mine maximal (k_1, k_2) -plexes, by designing effective techniques for search space pruning, and by developing efficient parallel implementation that effectively handles load balancing. Since small k -plexes are not statistically significant, it is common to add a lower bound of q vertices to the size of the k -plexes to emit. Despite the NP-hardness of our problem, these techniques allow maximal (k_1, k_2) -plex mining to scale to relatively large and dense graphs in real-world applications.

The main contributions of this paper are summarized as follows:

- We develop a recursive mining algorithm following the Bron-Kerbosch (BK) enumeration paradigm (Section III).
- We propose effective first- and second-order pruning rules to remove vertices and edges that cannot belong to any result, and design an efficient method to utilize them by an auxiliary structure (Section IV).
- Under the realistic assumption that $k_1, k_2 \leq \frac{q+1}{2}$, we show that the candidate vertices for (k_1, k_2) -plexes containing a vertex v is confined within 2 hops from v (ignoring edge directions), based on which we propose

an iterative two-hop pruning approach to further prune vertices (Section V).

- Two effective pruning techniques, look-ahead pruning and branch pruning, are further integrated to the BK enumeration process (Section VI).
- We propose a task-based parallel computing approach over our algorithm to achieve ideal speedup, integrated with a timeout mechanism to eliminate straggler tasks to allow effective load balancing (Section VII).
- Extensive experiments were conducted to demonstrate the effectiveness of the pruning techniques, and the scale-up capability of our parallelization.

The rest of this paper are organized as follows. Section II formally defines our notations and problem. Section III presents our recursive mining algorithm. Then, Section IV describes our first- and second-order pruning rules, Section V presents our iterative two-hop pruning approach, and Section VI introduces our look-ahead and branch pruning techniques. Following them, Section VII describes our task-based parallel computing approach which is integrated with a timeout mechanism to eliminate straggler tasks. Finally, Section VIII reports our experiments, Section IX reviews the related work, and Section X concludes this paper.

II. NOTATIONS AND PROBLEM DEFINITION

We first define some notations to facilitate the presentation. We consider a directed and unweighted simple graph $G = (V, E)$, where $n = |V|$ and $m = |E|$.

Given a vertex subset $V' \subseteq V$, for each vertex $v \in V$, we denote the set of outgoing (resp. incoming) neighbors of a vertex v in V' by $N_{V'}^+(v)$ (resp. $N_{V'}^-(v)$), and denote the out-degree (resp. indegree) of v in V' by $d_{V'}^+(v) = |N_{V'}^+(v)|$ (resp. $d_{V'}^-(v) = |N_{V'}^-(v)|$). We denote the bidirectional neighbors of v in V' by $N_{V'}^\pm(v) = N_{V'}^+(v) \cup N_{V'}^-(v)$. We omit the subscript V' when $V' = V$. We also define the concept of *non-neighbor*: a vertex u is not an outgoing (resp. incoming) neighbor of v , we say that u is a non-out-neighbor (resp. non-in-neighbor) of v . Accordingly, the set of non-out-neighbors (resp. non-in-neighbors) of v in V' is denoted by $\overline{N}_{V'}^+(v) = V' - N_{V'}^+(v)$ (resp. $\overline{N}_{V'}^-(v) = V' - N_{V'}^-(v)$), and we denote its cardinality by $\overline{d}_{V'}^+(v) = |\overline{N}_{V'}^+(v)|$ (resp. $\overline{d}_{V'}^-(v) = |\overline{N}_{V'}^-(v)|$).

Definition 1. $((k_1, k_2)$ -plex) A directed graph $g = (V_g, E_g)$ is a (k_1, k_2) -plex if g is connected, and for every vertex $v \in V_g$, we have $\overline{d}_{V_g}^+(v) \leq k_1$ and $\overline{d}_{V_g}^-(v) \leq k_2$.

Since small k -plexes are not statistically significant, it is common to add a lower bound to the size of the k -plexes to emit [3], [10], [11]. We also consider a size lower bound q , and only emit (k_1, k_2) -plexes with at least q vertices.

III. THE BRON-KERBOSCH ALGORITHM

Due to the hereditary property of k -plex (i.e., any induced subgraph of a k -plex is also a k -plex), we can mine maximal k -plexes using the Bron-Kerbosch algorithm [9]. Note that (k_1, k_2) -plex also satisfies the hereditary property:

Algorithm 1: The Bron-Kerbosch (BK) Algorithm

```

1 function BK( $G, k_1, k_2, P, C, X$ )
2   if  $C = \emptyset$  then
3     if  $X = \emptyset$  then
4       emit  $P$ 
5   else
6     for each  $v \in C$  do
7        $C \leftarrow C - \{v\}$ 
8        $C' \leftarrow \{v' \in C \mid P \cup \{v, v'\} \text{ is a } (k_1, k_2)\text{-plex}\}$ 
9        $X' \leftarrow \{v' \in X \mid P \cup \{v, v'\} \text{ is a } (k_1, k_2)\text{-plex}\}$ 
10      BK( $G, k_1, k_2, P \cup \{v\}, C', X'$ )
11       $X \leftarrow X \cup \{v\}$ 

```

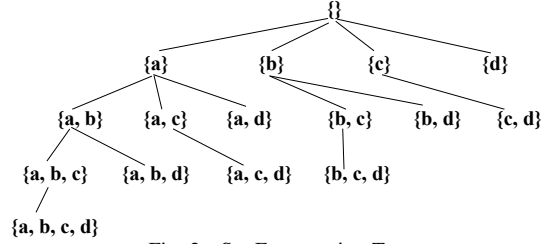


Fig. 2. Set-Enumeration Tree

Theorem 1. Given a (k_1, k_2) -plex P , any subset $P' \subseteq P$ is also a (k_1, k_2) -plex.

Due to page limit, we move all proofs into our online appendices [20]. The proof of Theorem 1 is given in Appendix A. As a result, we can extend the Bron-Kerbosch (BK) algorithm for mining maximal k -plexes (i.e., Algorithm 1 of [9]) to mine maximal (k_1, k_2) -plexes, as shown in Algorithm 1.

Specifically, three disjoint sets of vertices P , C , and X are maintained during the search, where (1) P is the set of vertices already included in the current subgraph, (2) C is the set of vertices that can extend P into a (k_1, k_2) -plex, and (3) X is the exclusive set used to check result maximality. This algorithm is recursive, and the recursion tree for a graph with four vertices a, b, c and d is illustrated in Figure 2, where each node is shown with set P .

BK maintains the invariant that for any vertex v in C and X , it can form a (k_1, k_2) -plex with P (otherwise, $P \cup \{v\}$ cannot be extended to a (k_1, k_2) -plex due to the hereditary property). This is guaranteed by Lines 8–9. The exclusive set X maintains those candidate vertices that have been considered by a previous tree branch. For example, consider node $\{a, c\}$ in Figure 2: since b has been used to extend $\{a\}$ into its sibling on the left, b will be added to X for the branch rooted at $\{a, c\}$. Later if, for example, $\{a, c, d\}$ is found to be a (k_1, k_2) -plex but $b \in X$, then Line 4 will not emit $\{a, c, d\}$ since adding b will produce a larger (k_1, k_2) -plex. When C and X are both empty (Lines 2–3), there are no further vertices that can be added to P , so P is a maximal (k_1, k_2) -plex.

Note that Lines 8–9 need to check if adding v to a (k_1, k_2) -plex P would produce a new (k_1, k_2) -plex. This can be easily checked if we incrementally maintain $d_P^+(\cdot)$ and $d_P^-(\cdot)$, as we can generalize Lemma 1 of [9] to show that $P \cup \{v\}$ is a (k_1, k_2) -plex if and only if (1) v is connected to all vertices u which satisfies $u \in P$ and $\overline{d}_P^-(u) = |P| - k_2$, (2) v is connected

from all vertices u which satisfies $u \in P$ and $d_P^+(u) = |P| - k_1$, and (3) $d_P^+(v) \geq |P| + 1 - k_1$ and $d_P^-(v) \geq |P| + 1 - k_2$.

IV. PRUNING BY SIZE LOWER BOUND

Since small k -plexes are not statistically significant, it is common to add a size lower bound to the k -plexes to emit, i.e., we only emit k -plexes with $\geq q$ vertices.

Note that instead of mining G directly, we can shrink G into its $(q - k_1, q - k_2)$ -core [21] for mining, which is the maximal subgraph $G' = (V', E')$ of G where every vertex has $d_{V'}^+(v) \geq q - k_1$ and $d_{V'}^-(v) \geq q - k_2$.

Theorem 2. *Given a graph $G = (V, E)$, all the (k_1, k_2) -plexes with at least q vertices must be contained in the $(q - k_1, q - k_2)$ -core of G .*

This is because for any vertex v in a (k_1, k_2) -plex P , $d_P^+(v) \geq |P| - k_1 \geq q - k_1$ and $d_P^-(v) \geq |P| - k_2 \geq q - k_2$. The $(q - k_1, q - k_2)$ -core can be computed using a peeling algorithm, which repeatedly removes vertices (and their adjacent edges) with $d^+(v) < q - k_1$ and $d^-(v) < q - k_2$ until there is no such vertex. We always shrink a graph G into its $(q - k_1, q - k_2)$ -core before mining.

We can also derive the second-order pruning rules as follows, which can be used to prune edges.

Theorem 3. *Let P be a (k_1, k_2) -plex with $|P| \geq q$. Then, for any two vertices $u, v \in P$, we have (i) if $(u, v) \notin E$, $|N_P^+(u) \cap N_P^-(v)| \geq q - k_1 - k_2 + 2$, (ii) otherwise, $|N_P^+(u) \cap N_P^-(v)| \geq q - k_1 - k_2$.*

The proof is in Appendix B [20]. Similarly, we have the following 3 rules:

Theorem 4. *Let P be a (k_1, k_2) -plex with $|P| \geq q$. Then, for any two vertices $u, v \in P$, we have (i) if $(u, v) \notin E$, $|N_P^+(u) \cap N_P^+(v)| \geq q - 2k_1 + 2$, (ii) otherwise, $|N_P^+(u) \cap N_P^+(v)| \geq q - 2k_1$.*

Theorem 5. *Let P be a (k_1, k_2) -plex with $|P| \geq q$. Then, for any two vertices $u, v \in P$, we have (i) if $(u, v) \notin E$, $|N_P^-(u) \cap N_P^-(v)| \geq q - k_1 - k_2 + 2$, (ii) otherwise, $|N_P^-(u) \cap N_P^-(v)| \geq q - k_1 - k_2$.*

Theorem 6. *Let P be a (k_1, k_2) -plex with $|P| \geq q$. Then, for any two vertices $u, v \in P$, we have (i) if $(u, v) \notin E$, $|N_P^-(u) \cap N_P^+(v)| \geq q - 2k_2 + 2$, (ii) otherwise, $|N_P^-(u) \cap N_P^+(v)| \geq q - 2k_2$.*

For simplicity, let us denote the number of common neighbors $|N^+(u) \cap N^-(v)|$ by $cn^{+-}(u, v)$, and we can similarly define $cn^{++}(u, v)$, $cn^{-+}(u, v)$ and $cn^{--}(u, v)$. To utilize second-order pruning rules above, we can iteratively remove edges (u, v) with $cn^{+-}(u, v) < q - k_1 - k_2$ or $cn^{++}(u, v) < q - 2k_1$ or $cn^{-+}(u, v) < q - k_1 - k_2$ or $cn^{--}(u, v) < q - 2k_2$. Note that the degree and common-neighbor counters can be initially computed for G and then incrementally maintained. The details are given in Appendix C [20].

Algorithm 2: *iterative_twohop_pruning*(v)

```

1  $O \leftarrow N^+(v) - N^\pm(v), \quad I \leftarrow N^-(v) - N^\pm(v)$ 
2 repeat
3    $S_O \leftarrow O \cup N^\pm(v), \quad S_I \leftarrow I \cup N^\pm(v)$ 
4    $O \leftarrow O \cap \bigcup_{w \in S_I} N^-(w), \quad I \leftarrow I \cap \bigcup_{w \in S_O} N^+(w)$ 
5 until neither  $O$  and  $I$  shrank in Line 4
6  $B \leftarrow \bigcup_{w \in S_O} N^+(w) \cap \bigcup_{w \in S_I} N^-(w) \cap$   

    $\bigcup_{w \in S_I} N^+(w) \cap \bigcup_{w \in S_O} N^-(w)$ 
7 return  $N^\pm(v) \cup O \cup I \cup B$ 

```

Algorithm 3: Enumerating with Initial Tasks

```

1 Prune  $G$  by first- and second-order pruning rules
2 Sort  $V$  by the degeneracy order as  $v_1 < v_2 < \dots < v_n$ 
3 for  $i = 1, 2, \dots, n - q + 1$  do
4    $P \leftarrow \{v_i\}$ 
5    $C \leftarrow \{v_{i+1}, \dots, v_n\} \cap \mathbb{B}(v_i)$ 
6    $X \leftarrow \{v_1, v_2, \dots, v_{i-1}\} \cap \mathbb{B}(v_i)$ 
7    $G_i \leftarrow$  the subgraph of  $G$  induced by  $\mathbb{B}(v_i) \cup \{v_i\}$ 
8   Prune  $G_i$  by first- and second-order pruning rules
9    $\text{BK}(G_i, k_1, k_2, P, C, X)$ 

```

V. TWO-HOP DIAMETER-BASED PRUNING

In Section IV, we shrink G by alternating edge peeling and vertex peeling. However, it is still inefficient to directly run the BK algorithm on the preprocessed G . We next show that it is possible to decompose the problem into mining over smaller subgraphs with diameter at most 2 (ignoring edge directions) with some reasonable assumptions on the values of k_1 and k_2 .

In k -plex mining, it is common to assume $q \geq 2k - 1$ which guarantees that the k -plex is connected with diameter of at most 2 [11], while a k -plex with $(2k - 2)$ vertices may be disconnected such as one consisting of two $(k - 1)$ -cliques.

We can extend this assumption to (k_1, k_2) -plex: to mine maximal (k_1, k_2) -plexes with at least q vertices, where $k_1, k_2 \leq \frac{q+1}{2}$. This is a reasonable assumption: for example, when q is merely 9, we can already allow k_1, k_2 to be as large as 5. This size lower bound enables a few search-space pruning opportunities.

Specifically, we can derive a tight vertex candidate set that can form (k_1, k_2) -plexes with a vertex v , denoted by $\mathbb{B}(v)$, which can be computed by Algorithm 2 which locates and removes unpromising vertices iteratively.

Theorem 7. *Let P be a (k_1, k_2) -plex where $k_1, k_2 \leq \frac{q+1}{2}$, and let v be a vertex in P , then for any other vertex $u \in P$, we have $u \in \mathbb{B}(v)$ where $\mathbb{B}(v)$ is computed by Algorithm 2.*

The proof is given in Appendix D [20].

Instead of calling the BK algorithm on G directly, we expand the first level of the set-enumeration tree to create initial tasks with $P = \{v_i\}$ for all $v_i \in V$, so that we can confine C and X to be within $\mathbb{B}(v_i)$, as shown in Algorithm 3.

Specifically, let $d_V^{\min}(v) = \min\{d_V^+(v), d_V^-(v)\}$, we can define a degeneracy order of the vertices using a peeling algorithm similar to its undirected graph counterpart: we remove the vertex with the smallest $d_V^{\min}(\cdot)$ from G each time

until all vertices are removed. Line 2 orders the vertices in this degeneracy order, so that all vertices in C have relatively small degrees (as real graphs have a small degeneracy), leading to initial tasks of more balanced workloads. Also, in Line 3, we do not need to consider $i \geq n - q + 2$ since such a task $P = \{v_i\}$ has $|P \cup C| < q$ which cannot generate a (k_1, k_2) -plex with more than q vertices. Finally, once C and X are properly constructed for each initial task $P = \{v_i\}$, we create a smaller subgraph G_i that only contains the relevant vertices and edges to expand $P = \{v_i\}$ in Line 7, and preprocess G_i to prune more vertices and edges by our first- and second-order pruning rules, and then call BK (i.e., Algorithm 1) using $\langle P, C, X \rangle$ on the smaller graph G_i to mine those maximal (k_1, k_2) -plexes that contain v_i as the smallest vertex (according to the degeneracy order of V). Note that in Line 4 of Algorithm 1, we now only emit P if $|P| \geq q$.

VI. BRANCH PRUNING

Look-Ahead Pruning. We next consider how to prune additional branches in the set-enumeration tree. Firstly, given $\langle P, C, X \rangle$ for enumeration, instead of extending P by vertices in C one by one, we can look ahead and check if $P \cup C$ produces a (k_1, k_2) -plex. If so, the entire set-enumeration subtree under P is pruned; moreover, if $X = \emptyset$, then $P \cup C$ is a maximal (k_1, k_2) -plex and should be emitted. We call this check as look-ahead pruning.

Branch Pruning. If $P \cup C$ is not a (k_1, k_2) -plex, then there must exist at least one vertex $v_p \in P \cup C$ with $d_{P \cup C}^+(v_p) < |P| + |C| - k_1$ or $d_{P \cup C}^-(v_p) < |P| + |C| - k_2$. Let us denote all such vertices by $M \subseteq P \cup C$. We next show that a vertex $v_p \in M \cap P$ (if it exists) can be used to reduce the # of branches when extending P .

Without loss of generality, assume that we pick a vertex $v_p \in M \cap P$ with $d_{P \cup C}^+(v_p) < |P| + |C| - k_1$ (the case for $d_{P \cup C}^-(v_p) < |P| + |C| - k_2$ is similar), and let us denote $N_C^+(v_p) = \{u_1, u_2, \dots, u_d\}$. Also, let $p = k_1 - \overline{d}_P^+(v_p)$, then $p < k_1$ and $p < d$. To see why $p < d$, assume that $p \geq d$ instead, then $k_1 - \overline{d}_P^+(v_p) \geq \overline{d}_C^+(v_p)$, so $\overline{d}_{P \cup C}^+(v_p) \leq k_1$ and hence $d_{P \cup C}^+(v_p) \geq |P| + |C| - k_1$, which contradicts with our assumption about v_p .

Since $p = k_1 - \overline{d}_P^+(v_p)$ is the largest number of non-out-neighbors of v_p in C that can still be added to P , we can generate $p+1 \leq d$ branches of BK calls. We build $\langle P', C', X' \rangle$ for the first branch as:

$$\langle P, C - \{u_1\}, X \cup \{u_1\} \rangle; \quad (1)$$

for branches $i = 2, \dots, p$ as:

$$\langle P \cup \{u_1, \dots, u_{i-1}\}, C - \{u_1, \dots, u_i\}, X \cup \{u_i\} \rangle; \quad (2)$$

for branch $(p+1)$ as:

$$\langle P \cup \{u_1, \dots, u_p\}, C - \{u_1, \dots, u_d\}, X \rangle. \quad (3)$$

Similarly, if v_p satisfies $d_{P \cup C}^-(v_p) < |P| + |C| - k_2$, then we can instead use $p = k_2 - \overline{d}_P^-(v_p)$ and $N_C^-(v_p) = \{u_1, u_2, \dots, u_d\}$ to generate the $p+1$ branches. If we can find

Algorithm 4: Improved BK Algorithm with Pruning

```

1 function BK2( $G, k_1, k_2, q, P, C, X$ )
2   if  $|P| + |C| < q$  then
3     return
4   if  $C = \emptyset$  then
5     if  $X = \emptyset$  then
6       emit  $P$ 
7   else
8      $v_p^+ \leftarrow$  a vertex with minimum  $d_{P \cup C}^+(\cdot)$  in  $P$ 
9      $v_p^- \leftarrow$  a vertex with minimum  $d_{P \cup C}^-(\cdot)$  in  $P$ 
10    if  $d_{P \cup C}^+(v_p^+) < |P| + |C| - k_1$  or
         $d_{P \cup C}^-(v_p^-) < |P| + |C| - k_2$  then
11      Pick  $v_p$  and  $\{u_1, u_2, \dots, u_d\}$ 
12      for  $i = 1, \dots, p+1$  do
13        Build the  $i^{\text{th}}$   $\langle P', C', X' \rangle$ 
14        if  $G[P']$  is a  $(k_1, k_2)$ -plex then
15           $C' \leftarrow \{v \in$ 
16             $C' \mid P' \cup \{v\}$  is a  $(k_1, k_2)$ -plex $\}$ 
17           $X' \leftarrow \{v \in$ 
18             $X' \mid P' \cup \{v\}$  is a  $(k_1, k_2)$ -plex $\}$ 
19          BK2( $G, k_1, k_2, q, P', C', X'$ )
20    else
21       $v_p^+ \leftarrow$  vertex with minimum  $d_{P \cup C}^+(\cdot)$  in  $C$ 
22       $v_p^- \leftarrow$  vertex with minimum  $d_{P \cup C}^-(\cdot)$  in  $C$ 
23      if  $d_{P \cup C}^+(v_p^+) \geq |P| + |C| - k_1$  and
           $d_{P \cup C}^-(v_p^-) \geq |P| + |C| - k_2$  then
24         $X' \leftarrow \{v \in$ 
25           $X \mid P \cup C \cup \{v\}$  is a  $(k_1, k_2)$ -plex $\}$ 
26        if  $X' = \emptyset$  then
27          emit  $P \cup C$ 
28    else
29      Pick  $v_p$ 
30      BK2( $G, k_1, k_2, q, P, C - \{v_p\}, X \cup \{v_p\}$ )
31       $C' \leftarrow \{v \in$ 
32         $C - \{v_p\} \mid P \cup \{v_p, v\}$  is a  $(k_1, k_2)$ -plex $\}$ 
33       $X' \leftarrow \{v \in$ 
34         $X \mid P \cup \{v_p, v\}$  is a  $(k_1, k_2)$ -plex $\}$ 
35      BK2( $G, k_1, k_2, q, P \cup \{v_p\}, C', X'$ )

```

from $M \cap P$ both a vertex v_p^+ with $d_{P \cup C}^+(v_p) < |P| + |C| - k_1$ and a vertex v_p^- with $d_{P \cup C}^-(v_p) < |P| + |C| - k_2$, then we set $p = \min \{k_1 - \overline{d}_P^+(v_p^+), k_2 - \overline{d}_P^-(v_p^-)\}$ and select $v_p = v_p^+$ or v_p^- accordingly, to keep the number of branches small.

The Improved BK Algorithm. We next present an improved BK algorithm that combines look-ahead pruning and branch pruning, as shown in Algorithm 4 (BK2). Note that Algorithm 3 Line 9 will now call BK2(.) instead of BK(.).

In Algorithm 4, Line 8 first picks v_p^+ as a vertex with minimum $d_{P \cup C}^+(\cdot)$ in P , and Line 9 picks v_p^- as a vertex with minimum $d_{P \cup C}^-(\cdot)$ in P . If the condition in Line 10 holds, then the entire $P \cup C$ cannot form a (k_1, k_2) -plex, branching is needed to recursively call BK2(.) (Lines 11–17). We select v_p and p as described previously, and generate $p+1$ branches of BK2(.) with $\langle P', C', X' \rangle$ as defined in Equations (1)–(3), which prunes $(d-p)$ branches.

If the condition at Line 10 does not hold, we check if $P \cup C$

can be (k_1, k_2) -plex (Lines 19–24). If the condition at Line 21 holds, for any vertex $v \in P \cup C$, we have $d_{P \cup C}^+(v) \geq |P| + |C| - k_1$ and $d_{P \cup C}^-(v) \geq |P| + |C| - k_2$, so $P \cup C$ is a (k_1, k_2) -plex to be emitted in Line 24 (note that $|P \cup C| \geq q$ as guaranteed by Lines 2–3). This is the look-ahead pruning.

If $P \cup C$ is not look-ahead pruned, branching is needed to recursively call BK2(.) (Lines 26–30). Specifically, we select $v_p = v_p^+$ or v_p^- with minimum value of $\{d_{P \cup C}^+(v_p^+), d_{P \cup C}^-(v_p^-)\}$. We divide into two branches in Lines 27–30, one without v_p in the result subgraphs (Line 27), and the other with v_p in the result subgraphs (Line 30). No branch is pruned in this case, and our choice of v_p allows a more workload-balanced set-enumeration tree since for high-degree vertices, their lower-degree neighbors tend to have been considered and removed from C , leading to a smaller fanout when branching further.

Our branching scheme results in reduced worst-case time complexity as compared to the trivial $\mathcal{O}^*(2^{|V|})$ bound.

Theorem 8. *Given a directed graph $G = (V, E)$, Algorithm 4 enumerates all (k_1, k_2) -plexes in $\mathcal{O}(|V|^2 \gamma_k^{|V|})$ where $k_1, k_2 \geq 1, k = \max(k_1, k_2)$ and γ_k is the largest real root of $x^{k+2} - 2x^{k+1} + 1 = 0$. For example, when $k = 2, 3, 4$ and 5 , $\gamma_k = 1.839, 1.928, 1.966$ and 1.984 , respectively.*

The proof is given in Appendix E [20].

Incremental Set and Degree Maintenance. Instead of creating new sets P', C' and X' as the inputs into each recursive call $\text{BK2}(G, k_1, k_2, q, P', C', X')$ (see Lines 17, 27 and 30), we propose to reuse the same containers for P, C and X throughout the recursive execution, by initializing their spaces at the very beginning, and populating them with proper elements for use in each recursion body. We describe the details in Appendix F [20].

VII. THE PARALLEL ALGORITHM

Recall that Algorithm 3 expands the first level of the set-enumeration tree to create initial tasks with $P = \{v_i\}$ for all $v_i \in V$, so that we can confine C and X to be within $\mathbb{B}(v)$. A straightforward approach to parallelize this algorithm is to add all the initial tasks into a shared task queue Q for the computing threads to fetch and process, so that different initial tasks can be concurrently processed by different CPU cores.

However, **Challenge 1:** an initial task may have a giant set-enumeration subtree, leading to the straggler problem. We, therefore, propose to use a timeout mechanism to let a thread decompose a long-running task into smaller tasks (rather than process it till the very end), and add these new tasks to Q so that they can be concurrently fetched and processed by the available threads. Each task runs by first recording its task start time t_0 , and then running BK2(.) with input $\langle P, C, X \rangle$ as shown in Algorithm 4. Before each recursive call of BK2(.) at Lines 17, 27 and 30 of Algorithm 4, we need to check if the current task times out, i.e., $t_{\text{cur}} - t_0 \geq \tau_{\text{time}}$, where t_{cur} is the current time and τ_{time} is the timeout threshold. If timeout occurs, each calling of BK2(.) for $\langle P', C', X' \rangle$ is not

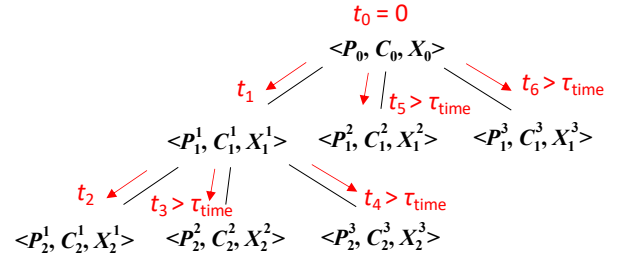


Fig. 3. Timeout-Based Task Decomposition

recursively executed by the current thread, but rather wrapped as an independent task and added to Q by the current thread.

Figure 3 illustrates how the timeout mechanism works for a task that computes BK2(.) for $\langle P_0, C_0, X_0 \rangle$. Recall that Algorithm 4 either creates $(p + 1)$ BK2-branches in the if-branch from Line 10, or creates two BK2-branches in the else-branch from Line 25, which essentially recursively expands the set-enumeration tree in depth-first order. As Figure 3 shows, let us assume that the depth-first processing reaches $\langle P_2^2, C_2^2, X_2^2 \rangle$ but finds $t_{\text{cur}} = t_3$ to time out. We then wrap $\langle P_2^2, C_2^2, X_2^2 \rangle$ as a new task and add it to Q , and when reaching the sibling $\langle P_2^1, C_2^1, X_2^1 \rangle$, we also add it as a new task to Q (since t_4 also times out). Since branching at this level is exhausted, Algorithm 4 would backtrack to the previous level and also add $\langle P_1^2, C_1^2, X_1^2 \rangle$ and $\langle P_1^3, C_1^3, X_1^3 \rangle$ as new tasks to Q (since t_5 and t_6 also time out). Note that the new tasks are created at different granularity and not over-decomposed.

This timeout mechanism guarantees that each task spends at least a duration of τ_{time} on the actual mining by backtracking (which does not materialize data such as $\langle P, C, X \rangle$) before dividing the remaining workloads into smaller tasks (which needs to materialize task data for tasks to be added to Q). The overhead of task creation is usually small, so it is outweighed by the benefit of improved load balancing due to the elimination of straggler tasks.

Challenge 2: As Figure 10 in Appendix F [20] shows, P, C and X are kept as global containers reused throughout the recursive execution, and each container has two arrays with capacity $|V|$. We now let each thread maintain its own global containers for P, C and X as well as buffers C'' and X'' , which are reused when the thread processes its fetched tasks one after another.

However, when adding a task that computes BK2(.) for $\langle P, C, X \rangle$ to Q , we cannot directly copy the two arrays $\text{list}[]$ and $\text{pos}[]$ shown in Figure 10, since $\text{pos}[]$ is sparse which basically requires copying the entire array of capacity $|V|$ that is very costly. We propose to only copy the elements in $\text{list}[]$ from the current thread's global container to the task object to be added to Q ; and when the task is fetched by a thread for processing, the thread reconstructs $\text{list}[]$ and $\text{pos}[]$ in its global containers from the saved elements. In this way, for the example shown in Figure 10, we only need to store the three elements 5, 3 and 8 with the created task, the cost of which is negligible.

Challenge 3: recall from Algorithm 3 that each initial task

requires the computation of $\mathbb{B}(v_i)$ to create the task subgraph G_i , which would be needed by all subsequent calls of BK2(.). In other words, all new tasks originated from this initial task would need access to G_i (and we say that these tasks form the task group of G_i), but it is inefficient to store G_i with each individual task in Q . Ideally, only one copy of G_i should be maintained and used by all tasks in the task group of G_i , and if all such tasks are completed, G_i should be timely released from the memory.

Moreover, it is inefficient to create initial tasks and add all of them to Q at the very beginning, due to the large memory overhead caused by maintaining the many task subgraphs G_i . Ideally, only a pool of active tasks are computed at any time to keep the memory usage bounded, and new initial tasks are only created when the current task pool is not sufficient to keep all CPU cores busy.

We address the above two requirements by letting each computing thread create and process the next unprocessed initial task one at a time. Each initial task with $P = \{v_i\}$ creates a task group that keeps G_i for use by its tasks. New tasks created due to the decomposition of an existing task in v_i 's task group are added to the same task group. A task group keeps track of its active tasks with its own queue, and is released from memory together with G_i when a thread finishes the last task in the group. When all initial tasks have been created, and a computing thread then becomes idle (so cannot take another new initial task), it will probe the task queue of an active task group (being processed by another thread) to steal its tasks for processing, to allow load balancing.

Note that instead of maintaining a global task queue Q , we are maintaining a set of active task groups, each of which maintains its own task queue and task subgraph. With M computing threads, there are at most M active task groups and hence M task subgraphs at any point of time, so the memory consumption is negligible. Also, this scheme maximizes data locality and hence cache hit rate, since a thread always finishes processing the tasks of a task group originated from v_i by repeatedly accessing G_i , before moving to the next task group.

VIII. EXPERIMENTS

This section reports our experiments. Our code has been released at <https://anonymous.4open.science/r/k1k2-plex-F6A9>.

Datasets. We use 15 real directed graphs, including 10 with small and medium size and 5 with large size (measured by # of vertices), as shown in Table I. These datasets span a wide range of graph sizes and categories: web crawl graphs *arabic-2005* [22], *uk-2005* [23], *it-2004* [24] and *webbase-2001* [25]; web graphs *web-google* [26] and *clue-web* [27]; social networks *bitcoin* [28], *soc-pokec* [29] and *epinions* [30]; contact network *math-overflow* [31]; co-purchasing network *amazon0505* [32]; communication network *wiki-talk* [33]; email network *email-euall* [34]; autonomous system *as-caida* [35]; and collaboration network *wiki-vote* [36].

In Table I, n (resp. m) denotes the number of vertices (resp. edges), ‘max d^+ ’ (resp. ‘max d^- ’) denotes the maximum outbound (resp. inbound) degree.

TABLE I
DATASETS

Datasets	n	m	max d^+	max d^-
bitcoin	5881	35,592	763	535
wiki-vote	7115	103,689	893	457
math-overflow	24,818	506,550	5931	5378
as-caida	26,475	53,381	2628	2628
epinions	75,879	508,837	1801	3035
email-auall	265,214	420,045	929	7631
amazon0505	410,236	3,356,824	10	2760
web-google	875,713	5,105,039	456	6326
soc-pokec	1,632,803	30,622,564	8763	13,733
wiki-talk	2,394,385	5,021,410	100,022	3311
arabic-2005	22,743,881	22,743,881	9905	575,618
uk-2005	39,459,925	936,364,282	5211	1,776,852
it-2004	41,290,648	1,027,474,947	9964	1,326,744
webbase-2001	115,554,441	854,809,761	3841	263,176
clue-web	147,925,593	454,072,685	1055	308,477

Experimental Setup. Our code is written in C++17 and compiled by g++ version 12.3.0 with optimization flag -O3. All the experiments are conducted on a platform with 24 cores (Intel Xeon Gold 6248R CPU 3.00GHz) and 256GB RAM. In our reports, the unit of time is seconds unless stated otherwise.

Our algorithm, denoted by **Ours**, integrates the various optimization techniques as described in Sections IV–VI. We also test various variants of our algorithms, assuming that we always first conduct the $(q - k_1, q - k_2)$ -core shrinking of G established by Theorem 2. Among the other techniques, we have:

- **Second-Order Pruning (SOP)** defined by Theorems 3–7, which removes edges (u, v) that cannot form a (k_1, k_2) -plex of size q due to insufficient common neighbors between u and v , along with cascaded removal of vertices and edges;
- **Iterative Pruning (ITP)** defined in Algorithm 2, which for each initial task with $P = \{v_i\}$, iteratively prunes vertices within 2 hops of v_i considering edge directions;
- **Branching (BRA)** defined in Algorithm 4 Lines 10–17, which skips impossible branches when a pivot vertex is selected from P ;
- **Look-ahead Pruning (LAP)** defined in Algorithm 4 Lines 19–24, which returns $P \cup C$ directly if it is a valid (k_1, k_2) -plex.

Since the concept of (k_1, k_2) -plex is newly proposed by us, there is no prior algorithms to compare with, hence we use variants of our algorithm as baselines where some of the above techniques are turned off. These baselines are

- **Base-0:** This naïve baseline only conducts the first-order pruning on the input graph as established by Theorem 2. No other techniques is applied when implementing Algorithm 3. The enumerative search of P is made on the entire graph after first-order pruning.
- **Base-1:** This baseline adds the SOP technique to Base-0;
- **Base-2:** This baseline improves Base-1 by conducting enumerative search for each initial task $P = \{v\}$ over a graph induced by the two-hop neighbors of v , which is computed naïvely ignoring the edge directions. Note that ITP is not yet applied;

TABLE II
THE RUNNING TIME OF MINING MAXIMAL (k_1, k_2) -PLEXES IN SERIAL
EXECUTION

Dataset	k_1	k_2	q	# of (k_1, k_2) -plexes	Running time (sec)				
					Base-0	Base-1	Base-2	Base-3	Ours
bitcoin	2	5	10	144	156.64	2.87	0.09	0.08	0.07
			12	0	47.35	0.43	0.02	0.02	0.02
	3	5	10	24261	>6 hrs	766.78	14.48	11.62	8.36
			12	286	9699.12	63.86	1.60	1.48	1.06
wiki-vote	3	5	10	474	>6 hrs	417.51	9.80	2.52	1.90
			12	0	>6 hrs	7.59	0.21	0.14	0.11
	4	5	10	30,222	>6 hrs	>6 hrs	7718.71	440.36	349.35
			12	94	>6 hrs	3872.87	100.47	20.22	16.66
mathoverflow	2	2	10	523,633	15,296.74	3758.61	133.00	129.30	28.84
			12	150,888	8815.69	1185.23	37.73	36.95	13.64
	2	3	10	1,762,917	>6 hrs	9775.45	472.32	463.00	63.44
			12	602,862	15,271.92	3995.21	141.70	140.04	32.72
as-caida	2	3	10	602,862	44.63	35.06	33.59	34.07	8.00
			15	23,314	1.63	0.68	0.56	1.23	0.94
	3	4	15	17,303	57.79	40.34	39.12	40.37	14.88
			20	0	29.04	0.79	0.37	0.34	0.20
epinions	2	3	10	773,095	>6 hrs	5165.51	157.60	149.90	25.14
			12	252,511	>6 hrs	1788.94	50.63	49.15	11.38
	3	4	12	24,599,029	>6 hrs	>6 hrs	7846.14	6343.36	2054.20
			15	2,382,084	>6 hrs	>6 hrs	1097.37	1008.13	382.83
email-euall	2	3	10	17,438	1684.69	171.80	3.46	2.93	1.74
			12	1442	895.40	44.45	1.04	0.95	0.63
	3	4	12	196,878	>6 hrs	8669.24	191.22	130.21	80.63
			15	1351	>6 hrs	723.05	17.70	15.75	9.24
amazon0505	3	5	10	4696	>6 hrs	>6 hrs	0.31	0.34	0.36
			12	24	>6 hrs	>6 hrs	0.13	0.14	0.15
	5	3	10	1532	>6 hrs	>6 hrs	0.19	0.21	0.22
			12	1	>6 hrs	>6 hrs	0.13	0.13	0.14
web-google	2	3	15	997	>6 hrs	>6 hrs	4075.22	4063.84	0.36
			20	57	>6 hrs	18,852.56	1013.29	1021.62	0.23
	3	4	15	1993	>6 hrs	>6 hrs	4057.13	4056.42	0.41
			20	61	>6 hrs	>6 hrs	1011.00	1010.43	0.25
soc-pokec	2	3	15	17,980	>6 hrs	4016.86	45.49	47.01	51.37
			20	17	>6 hrs	686.98	15.39	15.73	18.80
	3	3	15	508,884	>6 hrs	>6 hrs	213.41	205.30	109.69
			20	648	>6 hrs	2815.44	18.84	19.22	21.55
wiki-talk	2	2	15	22,219	>6 hrs	12,642.50	689.04	670.27	292.94
			20	0	>6 hrs	5040.90	197.85	194.52	75.14
	2	3	15	165,156	>6 hrs	>6 hrs	1928.46	1843.84	530.74
			20	0	>6 hrs	11,807.27	599.92	603.14	134.39

- **Base-3:** This baseline improves Base-1 by adding ITP, which further refines the set of two-hop neighbors of v , as compared with the one computed by Base-2.

All these baselines use BK (Algorithm 1) rather than BK2 (Algorithm 4) for enumerative search, while Ours improves Base-3 by using BK2 which additionally conducts branch pruning (BRA) and look-ahead pruning (LAP).

Performance of Serial Execution. We first compare the serial versions of Ours and the four baselines with parameters $(k_1, k_2) = 2-5$ and $q = 10-20$. The parameters are selected in accordance with the existing k -plex literature.

Table II reports the performance of our mining algorithms on the small and medium datasets. The execution time consists of the time for first-order and second-order pruning of the input graph, subgraph construction time for initial tasks, and (k_1, k_2) -plex enumeration time. We can see that Ours outperforms all the other baselines on all the datasets, with the exception in a few cases where Base-2 marginally outperforms Ours. These cases all have very short running time, meaning that the mining workloads are low and our techniques ITP, BRA and LAP cannot prune a lot of computations so the overhead does not pay off.

As Table II shows, Base-0 and Base-1 are not able to complete the mining of many datasets within 6 hours which we set as the job cutoff time, showing the importance of using

SOP and ITP for graph shrinking, so that enumerative search can be conducted with shorter adjacency lists. Even Base-2 and Base-3 can be several orders of magnitude slower than Ours (e.g., on web-google) if not many times slower, indicating that BRA and LAP in Ours are indispensable. To illustrate, on the web-google dataset with $(k_1, k_2) = (2, 3)$ and $q = 20$, Base-0 could not complete within the cutoff time, while Base-1, Base-2 and Base-3 are $81,965\times$, $4,404\times$ and $4,439\times$, respectively, slower than Ours. The large datasets from Table I are not included in Table II since they could not be solved in the cutoff time by any of the baselines, so Ours is the only choice.

We also study how the running time changes with different parameter values of (k_1, k_2) and q . Due to space limit, and without loss of generality, Figure 4 shows the results only for the first two medium datasets mathoverflow and as-caida. We do not include Base-0 since it is time-consuming and will make the other lines look horizontal and coinciding with each other near the horizontal axis. We can see that Ours is consistently the fastest for various values of (k_1, k_2) and q .

Performance of Parallel Execution and Scalability. We implemented the parallel version of Ours based on the description in Section VII. Table III reports the (k_1, k_2) -plex enumeration time of parallel Ours on large datasets in Table I (See Appendix H [20] for the results on all datasets) when running with 2, 4, 8, 16 and 32 threads, respectively, where we

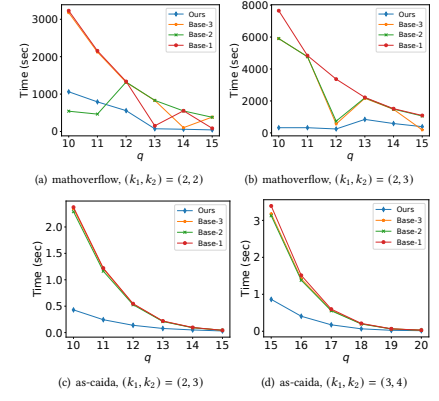


Fig. 4. The Running Time of Various Datasets for Different (k_1, k_2) when q Varies

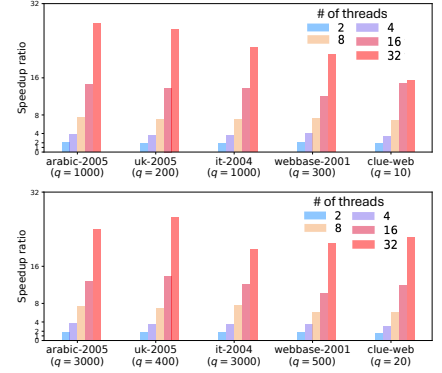


Fig. 5. The Speedup Ratio of Our Parallel Algorithm on the Five Large Networks

TABLE III
THE RUNNING TIME OF MINING MAXIMAL (k_1, k_2) -PLEXES IN PARALLEL EXECUTION

Dataset	k_1	k_2	q	# of (k_1, k_2) -plexes	Serial	2 Threads	4 Threads	8 Threads	16 Threads	32 Threads
arabic-2005	2	2	1000	106,895	3,906.70	1,897.35	1,017.31	527.40	269.04	140.58
	3	3	3000	3,500	111.60	60.20	29.56	15.10	8.69	4.64
uk-2005	2	2	200	117,255	301.09	162.70	84.11	43.34	21.82	11.38
	3	3	400	66,639	168.98	91.51	46.34	23.95	12.11	6.34
it-2004	2	2	1000	15,087	1,170.05	621.80	321.12	166.53	85.40	52.04
	3	3	3000	1,034	77.90	41.35	21.72	10.21	6.40	3.94
webbase-2001	2	2	300	327,882	646.41	320.84	164.46	90.02	53.56	30.75
	3	3	500	88,777	184.44	95.16	52.07	30	18.15	8.75
clue-web	2	2	10	1,215,977	26.05	14.15	7.66	3.82	1.75	1.69
	3	3	20	95,383	52.92	31.76	16.46	8.56	4.42	2.37

TABLE IV
ABLATION STUDY.

Dataset	k_1	k_2	q	Ours\SOP	Ours\ITP	Ours\BRA	Ours\LAP	Ours
bitcoin	2	5	10	92.57	0.07	0.08	0.07	0.06
			12	48.56	0.02	0.02	0.02	0.01
	3	5	10	98.05	10.37	11.49	8.82	8.35
			12	79.78	1.13	1.49	1.09	1.05
wiki-vote	3	5	10	1685.38	7.73	2.55	1.98	1.85
			12	1423.06	0.17	0.14	0.11	0.08
	4	5	10	2363.89	6192.11	447.41	369.35	349.32
			12	1859.31	81.95	20.68	17.53	16.63
arabic-2005	2	2	1000	8436.63	>6 hrs	>6 hrs	>6 hrs	3906.70
	3	3	3000	133.37	>6 hrs	215.30	>6 hrs	111.60
uk-2005	2	2	200	>6 hrs	386.27	>6 hrs	>6 hrs	301.09
	3	3	400	412.13	189.50	>6 hrs	>6 hrs	168.98

use the default timeout threshold $\tau_{time} = 0.1$ ms is adopted, which is tested to work consistently well on various datasets. As Table III shows, our parallel algorithm is efficient and scales up well with the number of CPU cores on all the datasets. Due to space limit, we show the speedup ratio of our parallel algorithm in Figure 5 only for the 5 large datasets. We can see that our parallel algorithm can achieve a speedup ratio of up to $27.80\times$ when with 32 threads.

Experiments on Ablation Study and Load Balancing. To verify the effectiveness of the various techniques we proposed, we conduct ablation study with Ours by disabling one of the technique at a time. This gives four variants Ours\SOP, Ours\ITP, Ours\BRA, Ours\LAP. For example, Ours\SOP is the variant when second-order pruning is turned off in Ours. Table IV shows the results of our ablation study with serial execution for the two small datasets and the first two large datasets. The other results are similar but omitted. It can be observed that Ours is the clear winner, and all of the techniques are beneficial. In particular, SOP is essential on the two small graphs to achieve orders of magnitude speedup, while ITP is also essential in a number of cases such as on wiki-vote with $k_1 = 4$ and $k_2 = 5$. The four techniques are even more important on large graphs since the program frequently runs beyond 6 hours even when missing one technique. In particular, LAP is essential to allow our four large-graph experiments to finish within 6 hours, followed by BRA for 3 experiments to be within 6 hours, then by ITP for 2 experiments and finally by SOP for 1 experiment. We can see that BK2 with BRA and LAP becomes more important than the graph pruning techniques SOP and ITP on large graphs, though SOP and ITP are more important on small graphs.

We also conducted experiments to study the impact of τ_{time} . The experimental results and detailed discussions can be found in Appendix G [20], where we observe that our default setting of $\tau_{time} = 0.1$ ms does work the best in general.

Case Study. We conducted a case study using the citation network from [37] which consists of papers chosen from ArnetMiner [38], [39] that fall in 10 topics such as Data Mining and Database Systems, as well as their citation relationship. Since the vertices are papers, but the citation relationship is one-way in time, it is improper to directly mine (k_1, k_2) -plexes on this network. We, therefore, preprocess this network to obtain an author citation network G , where an edge (u, v) is added if there exists a paper (co-)authored by u that cited another paper (co-)authored by v . In total, G contains 2,160 authors and 19,192 citation edges.

We mined $(2, 2)$ -plexes on G with $q = 8$, and obtained 118 result subgraphs in total in 33 milliseconds. Figure 6 (a) shows a result $(2, 2)$ -plex with 10 authors, where all authors are mutually cited except for Anthony Tomasic (AT), who cited Jennifer Widom (JW) but JW did not cite back. In other words, AT misses two outgoing links, one to JW and one to AT himself; and JW misses two incoming links, one from AT and one from JW herself. Apparently, this result is a valid $(2, 2)$ -plex. We can see that this subgraph corresponds to a community of renowned database researchers who frequently cite each others' works; in fact, Jennifer Widom was Jun Yang's PhD advisor, so they know each other.

Figure 6 (b) shows another result $(2, 2)$ -plex with 8 authors, where all authors are mutually cited except for Kyuseok Shim (KS), who cited Jong Soo Park (JSP) but JSP did not cite back. We can see that this subgraph corresponds to a community of

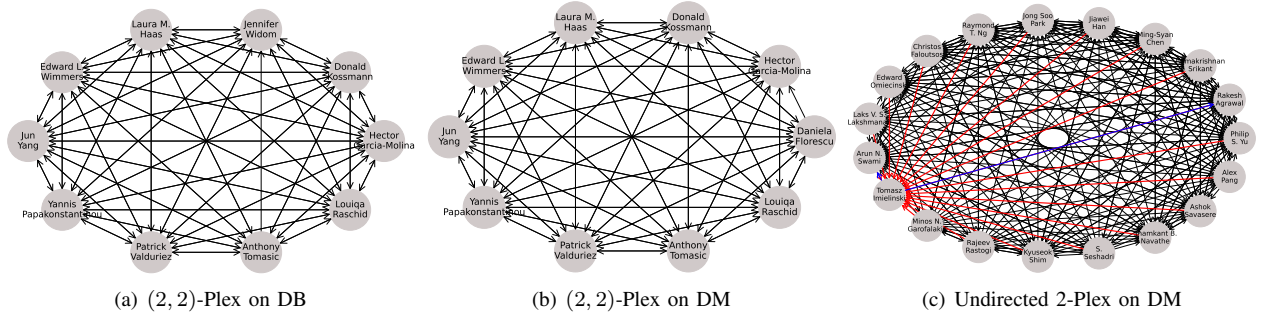


Fig. 6. Real Plexes in G

renowned data mining (DM) researchers who frequently cite each others' works.

An alternative method without using (k_1, k_2) -plex mining is to mine k -plexes from the undirected version of G (by making every edge bidirected). We mined 2-plexes on this graph with $q = 12$ and found 65,509 results. Figure 6 (c) shows the largest 2-plex with 19 authors, who are in the area of data mining. However, some authors in this 2-plex may miss many incoming or outgoing links. For example, Tomasz Imielinski has 18 incoming edges (highlighted in red) but only 2 outgoing edges (highlighted in blue). This is because k -plexes do not care about edge directions. Note that the 8 authors in Figure 6 (b) are also in the current 2-plex shown in Figure 6 (c), but this 8-author community cannot be found in the undirected case since it is not maximal given the current 2-plex. This shows the necessity of our (k_1, k_2) -plex formulation for finding highly cohesive directed subgraphs in real directed networks.

IX. RELATED WORK

We now review the works in three related areas: (1) maximal k -plex finding, (2) maximum k -plex finding, and (3) dense subgraphs in directed graphs.

Maximal k -Plex Finding. A k -plex is maximal, if it is not contained in any other k -plex in the input graph G . Berlowitz et al. [8] adopt a reverse search framework to enumerate maximal k -plexes. The key insight is that given a solution P , it is possible to find another solution by excluding some existing vertices from and including some new ones to H . The algorithm is more time-consuming than the Bron-Kerbosch algorithm [9], but provides a polynomial delay, so it is guaranteed to find some solutions in bounded time.

Many algorithms improve the Bron-Kerbosch algorithm by proposing effective search space pruning techniques. D2K [3] proposes a simple pivoting technique to cut useless branches, which generalizes the pivoting technique for maximal clique finding. FaPlexen [9] proposes a more advanced pivoting technique for branch pruning, and found that it competes favorably with D2K. FP [10] further proposes tight upper-bounding techniques to prune branches that cannot produce results of sufficient size (i.e., $\geq q$). For each vertex v , all the works reviewed so far treat all v 's two-hop neighbors as candidates to form valid k -plex results with v . ListPlex [11] proposes to enumerate subsets of the 2nd-hop neighbors of

v , denoted by S , to form an initial k -plex with v ; this k -plex $S \cup \{v\}$ is then expanded using candidates that are 1-hop neighbors of v . The reduced candidate size improves the enumeration performance both theoretically and empirically.

However, none of these works considered the context of directed graphs, which is the topic of this paper.

Maximum k -Plex Finding. A k -plex is maximum, if it has the most number of vertices among all k -plexes in the input graph G . Conte et al. [40] notice that any node of a k -plex P with $|P| \geq q$ is included in a clique of size at least $\lceil q/k \rceil$, which is used to prune invalid nodes. However, it is necessary to enumerate all maximal cliques which is expensive per se. To find a maximum k -plex, [40] uses binary search to guess the maximum k -plex size for vertex pruning, and then mines k -plexes on the pruned graph to see if such a maximum k -plex can be found. However, this approach may fail for multiple iterations before finding a maximum k -plex, so is less efficient than the branch-and-bound algorithms. BS [12] pioneers the brand-and-bound framework for finding a maximum k -plex with a number of pruning techniques.

For maximum k -plex finding, if the current maximum k -plex found is P , then we can prune any branch that cannot generate a k -plex with at least $|P| + 1$ vertices (i.e., upper bound $\leq |P| + 1$). Under the branch-and-bound enumeration framework, BnB [13] proposes upper bounds and pruning techniques based on deep structural analysis. KpLeX [14] proposes an upper bound based on vertex partitioning, and DisemKP [18] refines this partition-based upper bound based on the concept of distribution efficiency. Maplex [15] proposes an upper bound based on graph coloring which is later improved by RGB [41] and RelaxedGCB [19]. SeesawUB [19] further combines RelaxRGB and partition-based upper-bounding scheme to derive a tighter upper bound.

Among other works, kPlexS [16] proposes a technique called CTCP to prune the vertices and edges using the second-order property, and an upper bound that can utilize the second-order properties of non-adjacent vertex pairs. KPLeX [17] utilizes the enumerated subsets of the 2nd-hop neighbors of v to form initial k -plex with v for expansion, and solves the complementary d -bounded-degree-deletion (d -BDD) problem during branch-and-bound enumeration. KPLeXT [42] improves the time complexity for maximum k -plex computation with a new branching method.

Unlike these works that aim to find one largest k -plex, we

find large maximal results in a directed graph that can be in various regions of the graph and hence more diversified.

Dense Subgraphs in Directed Graphs. Although many real networks are directed, research on directed dense subgraphs lags behind. Only recently, the idea of k -core has been generalized to directed graphs, called D-core [43], [44]. However, this work focuses on finding a D-core that is strongly connected and contains a particular query vertex. DDS [45], [46] finds the densest subgraph from a large directed graph, which is defined as a pair of vertex sets (S, T) that maximizes the density of edges from vertices in S to vertices in T . However, it only cares about the outbound (resp. inbound) edge density of vertices in S (resp. T), rather than connectivity in both directions for every vertex in a subgraph like in our setting. Finally, [21] extends γ -quasi-cliques [47] from the context of undirected graphs to directed graphs, by proposing the concept of (γ_1, γ_2) -quasi-cliques. However, quasi-cliques define subgraph cohesiveness based on the ratio of vertex degree to the subgraph size, rather than the number of allowable missing neighbors as in k -plexes, so the problem is different.

X. CONCLUSIONS

We formulated the concept of (k_1, k_2) -plexes on directed graphs, and proposed an efficient recursive algorithm following the Bron-Kerbosch enumeration paradigm. The algorithm is integrated with search space pruning techniques including first- and second-order pruning, iterative pruning of 2-hop neighborhood of individual vertices, branch pruning and look-ahead pruning. Our algorithm is parallelized following a task-based framework with effective load balancing. Extensive experiments demonstrated the scalability of our algorithm on large directed graphs, the necessity of our pruning techniques, and the scale-up capability of our parallel implementation.

REFERENCES

- [1] S. B. Seidman and B. L. Foster, "A graph-theoretic generalization of the clique concept," *Journal of Mathematical sociology*, vol. 6, no. 1, pp. 139–154, 1978.
- [2] J. Patillo, N. Youssef, and S. Butenko, "On clique relaxation models in network analysis," *European Journal of Operational Research*, vol. 226, no. 1, pp. 9–18, 2013.
- [3] A. Conte, T. D. Matteis, D. D. Sensi, R. Grossi, A. Marino, and L. Versari, "D2K: scalable community detection in massive networks via small-diameter k -plexes," in *KDD*. ACM, 2018, pp. 1272–1281.
- [4] K. Y. Yeung, M. Medvedovic, and R. E. Bumgarner, "From co-expression to co-regulation: how many microarray experiments do we need?" *Genome biology*, vol. 5, no. 7, pp. 1–11, 2004.
- [5] B. K. Tanner, G. Warner, H. Stern, and S. Olechowski, "Koobface: The evolution of the social botnet," in *eCrime*. IEEE, 2010, pp. 1–10.
- [6] J. M. Lewis and M. Yannakakis, "The node-deletion problem for hereditary properties is np-complete," *J. Comput. Syst. Sci.*, vol. 20, no. 2, pp. 219–230, 1980.
- [7] B. Balasundaram, S. Butenko, and I. V. Hicks, "Clique relaxations in social network analysis: The maximum k -plex problem," *Oper. Res.*, vol. 59, no. 1, pp. 133–142, 2011.
- [8] D. Berlowitz, S. Cohen, and B. Kimelfeld, "Efficient enumeration of maximal k -plexes," in *SIGMOD*. ACM, 2015, pp. 431–444.
- [9] Y. Zhou, J. Xu, Z. Guo, M. Xiao, and Y. Jin, "Enumerating maximal k -plexes with worst-case time guarantee," in *AAAI*. AAAI, 2020, pp. 2442–2449.
- [10] Q. Dai, R. Li, H. Qin, M. Liao, and G. Wang, "Scaling up maximal k -plex enumeration," in *CIKM*. ACM, 2022, pp. 345–354.
- [11] Z. Wang, Y. Zhou, M. Xiao, and B. Khossainov, "Listing maximal k -plexes in large real-world graphs," in *WWW*. ACM, 2022, pp. 1517–1527.
- [12] M. Xiao, W. Lin, Y. Dai, and Y. Zeng, "A fast algorithm to compute maximum k -plexes in social network analysis," in *AAAI*. AAAI Press, 2017, pp. 919–925.
- [13] J. Gao, J. Chen, M. Yin, R. Chen, and Y. Wang, "An exact algorithm for maximum k -plexes in massive graphs," in *IJCAI*. ijcai.org, 2018, pp. 1449–1455.
- [14] H. Jiang, D. Zhu, Z. Xie, S. Yao, and Z. Fu, "A new upper bound based on vertex partitioning for the maximum k -plex problem," in *IJCAI*. ijcai.org, 2021, pp. 1689–1696.
- [15] Y. Zhou, S. Hu, M. Xiao, and Z. Fu, "Improving maximum k -plex solver via second-order reduction and graph color bounding," in *AAAI*. AAAI Press, 2021, pp. 12 453–12 460.
- [16] L. Chang, M. Xu, and D. Strash, "Efficient maximum k -plex computation over large sparse graphs," *Proc. VLDB Endow.*, vol. 16, no. 2, pp. 127–139, 2022. [Online]. Available: <https://www.vldb.org/pvldb/vol16/p127-chang.pdf>
- [17] Z. Wang, Y. Zhou, C. Luo, and M. Xiao, "A fast maximum k -plex algorithm parameterized by the degeneracy gap," in *IJCAI*. ijcai.org, 2023, pp. 5648–5656.
- [18] H. Jiang, F. Xu, Z. Zheng, B. Wang, and W. Zhou, "A refined upper bound and inprocessing for the maximum k -plex problem," in *IJCAI*. ijcai.org, 2023, pp. 5613–5621.
- [19] J. Zheng, M. Jin, and K. He, "Two new upper bounds for the maximum k -plex problem," 2023.
- [20] "Online Appendices," <https://github.com/akhlaqueak/k1k2-plex/appendix.pdf>.
- [21] G. Guo, D. Yan, L. Yuan, J. Khalil, C. Long, Z. Jiang, and Y. Zhou, "Maximal directed quasi-clique mining," in *ICDE*. IEEE, 2022, pp. 1900–1913.
- [22] "arabic-2005," <https://sparse.tamu.edu/LAW/arabic-2005>.
- [23] "uk-2005," <https://sparse.tamu.edu/LAW/uk-2005>.
- [24] "it-2004," <https://sparse.tamu.edu/LAW/it-2004>.
- [25] "webbase-2001," <https://sparse.tamu.edu/LAW/webbase-2001>.
- [26] "web-google," <https://snap.stanford.edu/data/web-Google.html>.
- [27] "clue-web," <https://networkrepository.com/web-ClueWeb09-50m.php>.
- [28] "bitcoin," <http://konect.cc/networks/soc-sign-bitcoinot/>.
- [29] "soc-pokec," <https://snap.stanford.edu/data/wiki-Pokec.html>.
- [30] "epinions," <http://konect.cc/networks/soc-Epinions1/>.
- [31] "math-overflow," <http://konect.cc/networks/sx-mathoverflow/>.
- [32] "amazon0505," <https://snap.stanford.edu/data/amazon0505.html>.
- [33] "wiki-talk," <https://snap.stanford.edu/data/wiki-Talk.html>.
- [34] "email-auall," <https://snap.stanford.edu/data/email-EuAll.html>.
- [35] "as-caida," <https://snap.stanford.edu/data/as-Caida.html>.
- [36] "wiki-vote," <https://snap.stanford.edu/data/wiki-Vote.html>.
- [37] "ArnetMiner Citation Network," <https://lfs.aminer.cn/lab-datasets/soinf/>.
- [38] J. Tang, J. Zhang, L. Yao, J. Li, L. Zhang, and Z. Su, "Arnetminer: extraction and mining of academic social networks," in *KDD*. ACM, 2008, pp. 990–998.
- [39] J. Tang, J. Sun, C. Wang, and Z. Yang, "Social influence analysis in large-scale networks," in *KDD*. ACM, 2009, pp. 807–816.
- [40] A. Conte, D. Firmani, C. Mordente, M. Patrignani, and R. Torlone, "Fast enumeration of large k -plexes," in *KDD*. ACM, 2017, pp. 115–124.
- [41] J. Zheng, M. Jin, Y. Jin, and K. He, "Relaxed graph color bound for the maximum k -plex problem," *CoRR*, vol. abs/2301.07300, 2023.
- [42] L. Chang and K. Yao, "Maximum k -plex computation: Theory and practice," *Proc. ACM Manag. Data*, vol. 2, no. 1, pp. 63:1–63:26, 2024.
- [43] C. Giatsidis, D. M. Thilikos, and M. Vazirgiannis, "D-cores: Measuring collaboration of directed graphs based on degeneracy," in *ICDM*. IEEE Computer Society, 2011, pp. 201–210.
- [44] Y. Fang, Z. Wang, R. Cheng, H. Wang, and J. Hu, "Effective and efficient community search over large directed graphs," *IEEE Trans. Knowl. Data Eng.*, vol. 31, no. 11, pp. 2093–2107, 2019.
- [45] C. Ma, Y. Fang, R. Cheng, L. V. S. Lakshmanan, W. Zhang, and X. Lin, "Efficient algorithms for densest subgraph discovery on large directed graphs," in *SIGMOD Conference*. ACM, 2020, pp. 1051–1066.
- [46] —, "On directed densest subgraph discovery," *ACM Trans. Database Syst.*, vol. 46, no. 4, pp. 13:1–13:45, 2021.
- [47] G. Liu and L. Wong, "Effective pruning techniques for mining quasi-cliques," in *ECML/PKDD*, ser. Lecture Notes in Computer Science, vol. 5212. Springer, 2008, pp. 33–49.

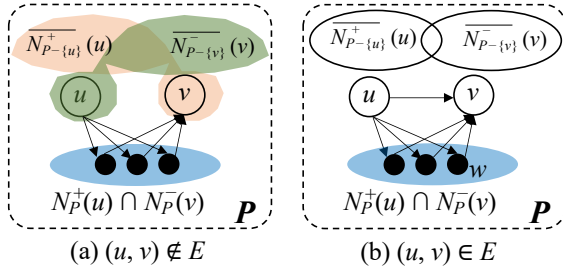


Fig. 7. Second-Order Pruning

APPENDIX A PROOF OF THEOREM 1

Proof. For any $v \in P'$, we have $v \in P$ and since P is a (k_1, k_2) -plex, we have $\overline{d_P^+}(v) = |\overline{N_P^+}(v)| \leq k_1$ and $\overline{d_P^-}(v) = |\overline{N_P^-}(v)| \leq k_2$. Since $\overline{N_{P'}^+}(u) \subseteq \overline{N_P^+}(u)$ and $\overline{N_{P'}^-}(u) \subseteq \overline{N_P^-}(u)$, we have $\overline{d_{P'}^+}(v) = |\overline{N_{P'}^+}(v)| \leq k_1$ and $\overline{d_{P'}^-}(v) = |\overline{N_{P'}^-}(v)| \leq k_2$, so P' is also a (k_1, k_2) -plex. $\square$

APPENDIX B PROOF OF THEOREM 3

Proof. This can be seen from Figure 7. Since P is a (k_1, k_2) -plex, we have $\overline{d_{P-\{u\}}^+}(u) \leq k_1 - 1$ (since $\overline{d_P^+}(u) \leq k_1$) and $\overline{d_{P-\{v\}}^-}(v) \leq k_2 - 1$ (since $\overline{d_P^-}(v) \leq k_2$).

In Case (a) where $(u, v) \notin E$, any vertex $w \in P$ can only fall in the following 3 scenarios: (1) $w \in \overline{N_{P-\{u\}}^+}(u)$ (which contains v), (2) $w \in \overline{N_{P-\{v\}}^-}(v)$ (which contains u), and (3) $w \in N_P^+(u) \cap N_P^-(v)$. Note that w may be in both (1) and (2). So we have:

$$\begin{aligned} |P| &= |\overline{N_P^+}(u) \cap \overline{N_P^-}(v)| + |\overline{N_{P-\{u\}}^+}(u) \cup \overline{N_{P-\{v\}}^-}(v)| \\ &\leq |\overline{N_P^+}(u) \cap \overline{N_P^-}(v)| + |\overline{N_{P-\{u\}}^+}(u)| + |\overline{N_{P-\{v\}}^-}(v)| \\ &\leq |\overline{N_P^+}(u) \cap \overline{N_P^-}(v)| + (k_1 - 1) + (k_2 - 1) \\ &= |\overline{N_P^+}(u) \cap \overline{N_P^-}(v)| + k_1 + k_2 - 2, \end{aligned}$$

so $|\overline{N_P^+}(u) \cap \overline{N_P^-}(v)| \geq |P| - k_1 - k_2 + 2 \geq q - k_1 - k_2 + 2$.

In Case (b) where $(u, v) \in E$, any vertex $w \in P$ can only be in one of the following 4 scenarios: (1) $w = u$, (2) $w = v$, (3) $w \in N_P^+(u) \cap N_P^-(v)$, and (4) $w \in \overline{N_{P-\{u\}}^+}(u) \cup \overline{N_{P-\{v\}}^-}(v)$, so:

$$\begin{aligned} |P| &= 2 + |\overline{N_P^+}(u) \cap \overline{N_P^-}(v)| + |\overline{N_{P-\{u\}}^+}(u) \cup \overline{N_{P-\{v\}}^-}(v)| \\ &\leq 2 + |\overline{N_P^+}(u) \cap \overline{N_P^-}(v)| + |\overline{N_{P-\{u\}}^+}(u)| + |\overline{N_{P-\{v\}}^-}(v)| \\ &\leq 2 + |\overline{N_P^+}(u) \cap \overline{N_P^-}(v)| + (k_1 - 1) + (k_2 - 1) \\ &= |\overline{N_P^+}(u) \cap \overline{N_P^-}(v)| + k_1 + k_2, \end{aligned}$$

so $|\overline{N_P^+}(u) \cap \overline{N_P^-}(v)| \geq |P| - k_1 - k_2 \geq q - k_1 - k_2$. $\square$

APPENDIX C COUNTER COMPUTATION

Initial Counter Computation. Figure 8 shows how we compute the initial values of the counters. Specifically, consider

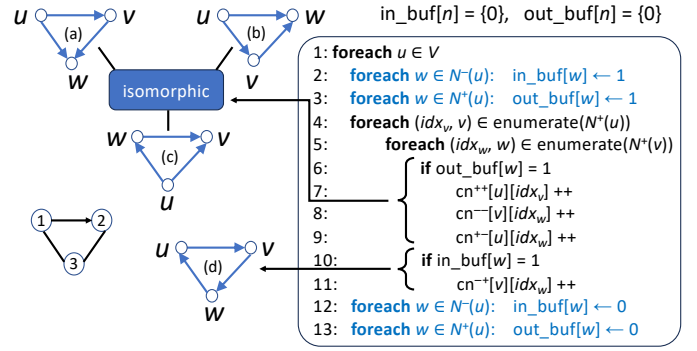


Fig. 8. Initial Counter Computation

an arbitrary (starting) edge (v_1, v_2) , then we can obtain a triangle $\Delta v_1 v_2 v_3$ with four possible edge orientations (a)–(d) as shown on the left of Figure 8. Note that the three acyclic configurations are isomorphic: each triangle of type (b) (resp. (c)) has a one-to-one mapping to a triangle of type (a), so we just need to enumerate triangles of type (a) to avoid redundancy.

Figure 8 shows this enumeration where $v \in N^+(u)$ (Line 4) and $w \in N^+(v)$ (Line 5), and Line 6 checks if $w \in N^+(u)$. Note that for each vertex u , we save its counters, such as $\text{cn}^{++}(u, :)$, also sparsely to be space-efficient. For each position idx in the adjacency list $N^+(u)$ such that $N^+(u)[idx] = v$, we save $\text{cn}^{++}[u][idx] = \text{cn}^{++}(u, v)$.

If Line 6 finds that $w \in N^+(u)$, we increment cn^{++} , cn^{--} , and cn^{+-} for the triangle's three edges properly. To avoid repeated binary search when checking if $w \in N^+(u)$ in Line 6, we use a bitmap out_bf of size $|V|$ that is initialized as all zeros. When we process a vertex $u \in V$ in Line 1, we first set the bits of all the out-neighbors in $N^+(u)$ to be 1 in Line 3, so that Line 6 can be implemented as a simple bit check. These bits are cleared in Line 13 before we continue to process the next vertex $u \in V$ in Line 1.

The cyclic triangle of type (d) increments cn^{+-} . In Line 10, if we find $w \in N^-(u)$, we increment $\text{cn}^{+-}(v, w)$ only since $\text{cn}^{+-}(w, u)$ (resp. $\text{cn}^{+-}(u, v)$) will be incremented when we enumerate from starting vertex v (resp. w) in Line 1. $\square$

Incremental Counter Maintenance. Let us consider edge peeling first. When we remove an edge (u, v) , (1) we decrement $d^+(u)$ and $d^-(v)$. (2) For each $w \in N^+(u) \cap N^-(v)$, we decrement $\text{cn}^{++}(u, w)$ and $\text{cn}^{--}(w, v)$ (see Figure 7(b) to get the intuition). Similarly, (3) for each $w \in N^+(u) \cap N^+(v)$, we decrement $\text{cn}^{+-}(u, w)$ and $\text{cn}^{--}(v, w)$; (4) for each $w \in N^-(u) \cap N^+(v)$, we decrement $\text{cn}^{+-}(w, u)$ and $\text{cn}^{+-}(v, w)$; (5) for each $w \in N^-(u) \cap N^-(v)$, we decrement $\text{cn}^{++}(w, u)$ and $\text{cn}^{++}(w, v)$.

Whenever the decrement of a common-neighbor counter for an edge e causes its value to be below the threshold stipulated by Theorems 3–6, e is added to an edge queue Q_e for further peeling. Vertices whose degrees get decremented below the thresholds stipulated by Theorem 2 are also added to a vertex queue Q_v . When edge peeling completes (i.e., $Q_e = \emptyset$), we conduct vertex peeling starting from the vertices in Q_v .

The removal of each vertex w in Q_v will also incrementally decrement the degrees (resp. common-neighbor counters) of w 's neighbors (resp. neighbor-pairs), for which we omit the details. Once vertex peeling completes, new edges may have been added to Q_e so we start another round of edge peeling. We alternate between edge peeling and vertex peeling until both Q_e and Q_v are empty. This process is similar to the CTCP algorithm (Algorithm 3) of [16] in the k -plex context. $\square$

Note that the above process of shrinking G by alternating edge peeling and vertex peeling is executed on top of the $(q - k_1, q - k_2)$ -core of G which is initially computed by a vertex peeling algorithm, since the latter is more efficient in pruning unpromising vertices as a preprocessing.

APPENDIX D PROOF OF THEOREM 7

Proof. Consider any two vertices u, v in a (k_1, k_2) -plex P where $k_1, k_2 \leq \frac{q+1}{2}$, we show that they cannot be more than 2 hops apart (i.e., cannot fall out of the 6 cases in Figure 9).

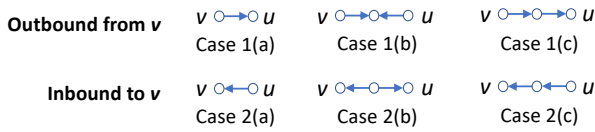


Fig. 9. Cases for Two-Hop Diameter Upper Bound

Without loss of generality, we only consider the path from v to u where the first edge is outbound from v , i.e., Cases 1(a)–(c). Cases 2(a)–(c) are symmetric and can be similarly proved.

If v directly points to u , we are done since Case 1(a) occurs. Now assume that edge $(v, u) \notin E$, and we show that:

- **Case (I):** $(u, v) \notin E$, then both Case 1(b) and Case 1(c) should be satisfied. (i) **We first prove Case 1(b).** Note that $u \notin N^+(v)$ and $v \notin N^+(u)$. Since $k_1 \leq \frac{q+1}{2} \leq \frac{|P|+1}{2}$, u and v each points to at least $|P| - k_1 \geq \frac{|P|-1}{2}$ other vertices in P , so they must share an out-neighbor; otherwise, there exist $2 \cdot \frac{|P|-1}{2} = |P| - 1$ vertices other than u and v , leading to a contradiction since there will be at least $(|P| + 1)$ vertices in P when adding u and v . (ii) **We next prove Case 1(c).** Note that $v \notin N^-(u)$ and $u \notin N^-(v)$. Since $k_1 \leq \frac{q+1}{2} \leq \frac{|P|+1}{2}$, v points to at least $|P| - k_1 \geq \frac{|P|-1}{2}$ other vertices in P ; also since $k_2 \leq \frac{q+1}{2} \leq \frac{|P|+1}{2}$, u is pointed to by at least $|P| - k_2 \geq \frac{|P|-1}{2}$ other vertices in P . So, $N^+(v)$ and $N^-(u)$ must intersect as illustrated by Figure 9 Case 1(c); otherwise, there will be $(|P| + 1)$ vertices in P when adding u and v .
- **Case (II):** $(u, v) \in E$, then Case 1(c) should be satisfied. The proof is the same as (ii) above. Note that we cannot guarantee Case 1(b) anymore, since $v \in N^+(u)$, i.e., v can be one of the at least $|P| - k_1 \geq \frac{|P|-1}{2}$ neighbors of u , invalidating the prove for (i) above.

Symmetrically, consider the path from v to u where the first edge is inbound to v , i.e., Cases 2(a)–(c). If u directly points to v , we are done since Case 2(a) occurs. If edge $(u, v) \notin E$:

- **Case (III):** edge (v, u) does not exist in G , then both Case 2(b) and Case 2(c) should be satisfied. The proof is symmetric to Case (I) above and thus omitted.
- **Case (IV):** edge (v, u) exists in G , then Case 2(c) should be satisfied. The proof is symmetric to Case (II) above.

Putting the above discussions together, we obtain the following 4 cases, for each of which we explain how to exclude an impossible candidate u from $ext(S)$ given a vertex $v \in S$, where $ext(S)$ denotes the set of candidate vertices that can extend S into a valid (k_1, k_2) -plex.

- **Case A:** $(v, u) \in E$ and $(u, v) \in E$. In this case, we always have $u \in ext(S)$.
- **Case B:** $(v, u) \notin E$ and $(u, v) \in E$. Based on Case (II) above, we have $u \in ext(S)$ only if a path $u \leftarrow w \leftarrow v$ exists in G for some $w \in V$ ($w \neq u, v$).
- **Case C:** $(v, u) \in E$ and $(u, v) \notin E$. Based on Case (IV) above, we have $u \in ext(S)$ only if a path $u \rightarrow w \rightarrow v$ exists in G for some $w \in V$ ($w \neq u, v$).
- **Case D:** $(v, u) \notin E$ and $(u, v) \notin E$. Based on Case (I) above, we have Condition (C1): $u \in ext(S)$ only if both Case 1(b) and Case 1(c) in Figure 9 are satisfied. Similarly, based on Case (III) above, we have Condition (C2): $u \in ext(S)$ only if both Case 2(b) and Case 2(c) are satisfied. Combining both conditions, $u \in ext(S)$ only if there exist $w_1, w_2, w_3, w_4 \in V - \{u, v\}$ such that $u \leftarrow w_1 \leftarrow v$ and $u \leftarrow w_2 \rightarrow v$ and $u \rightarrow w_3 \leftarrow v$ and $u \rightarrow w_4 \rightarrow v$.

Once we have applied the above rules to prune $ext(S)$ to exclude invalid candidates u , let us abuse the notation to use G again to denote the resulting graph induced by $S \cup ext(S)$ after pruning. Note that we can apply this diameter-based pruning on the pruned G again, since some vertex w in Case B (resp. Case C) could have been pruned by Case C (resp. Case B) in the previous iteration, causing some required paths to disappear, further invalidating more vertices u from $ext(S)$. This pruning can be iteratively run over G .

Based on the above idea, Algorithm 2 computes the set of vertices in $ext(S)$ that are not 2-hop pruned by a vertex $v \in S$. Specifically, Line 1 computes O (resp. I) as the set of v 's out-neighbors (resp. in-neighbors) u that belong to Case B (resp. Case C). Then, Line 3 recovers S_O (resp. S_I) as the set of v 's all non-pruned out-neighbors (resp. in-neighbors) w in Case B (resp. Case C) with path $v \rightarrow w \rightarrow u$ (resp. $v \leftarrow w \leftarrow u$). Note that $N^\pm(v) \subseteq ext(S)$ based on Case A so its vertices cannot be further pruned, so the iterative pruning is contributed by the shrink of sets O and I .

Next, Line 4 prunes away those vertices $u \in O$ (resp. $u \in I$) that cannot find a path $u \rightarrow w$ (resp. $u \leftarrow w$) for some non-pruned $w \in N^-(v)$ (resp. $w \in N^+(v)$), which is based on Case C (resp. Case B). Note that if O or I shrinks in Line 4, Line 5 will trigger another iteration of pruning. When the loop of Lines 2–5 exits, we have O (resp. I) being the remaining vertices $u \in ext(S)$ in Case B (resp. Case C) after iterative pruning. Finally, Line 6 computes the set B of vertices where u satisfies Case D w.r.t. v , and Line 7 unions the 4 disjoint

candidate sets that correspond to Cases A–D, respectively, to obtain the final 2-hop pruned $\text{ext}(S)$ for a vertex $v \in S$. We denote this set as $\mathbb{B}(v)$, which is returned by Line 7. $\square$

APPENDIX E COMPLEXITY ANALYSIS

Given a directed graph $G = (V, E)$, we evaluate the worst-case running time of Algorithm 4. Let $T(m, n)$ be the worst-case running time of function $\text{BK2}(G, k_1, k_2, q, P, C, X)$ where $m = |C|$ and $n = |P \cup C|$, and we define $k = \max(k_1, k_2)$. Next, we analyze $T(m, n)$ by considering four different cases:

I. When $C = \emptyset$ (Lines 4–6), then

$$T(m, n) = T(0, n) \leq c_1, \quad (4)$$

where c_1 is a constant.

II. When $P \cup C$ is not a (k_1, k_2) -plex and the pivot vertex is in P (Lines 8–17). Here, $p+1$ branches are created each with a smaller candidate set C' . Before each call the sets C' and X' are further shrunk in Lines 15–16 which takes time $O(|V|^2)$. The worst-case running time of this case is, therefore

$$T(m, n) \leq (p+1)c_2|V|^2 + \sum_{i=1}^p T(m-i, n-1) + T(m-d, n-d+p), \quad (5)$$

where c_2 is a constant.

III. When $P \cup C$ is a (k_1, k_2) -plex (Lines 19–24), then no further branches are created, and the cost in this case is only incurred by updating X' in Line 22, which is

$$T(m, n) \leq c_2|V|^2 \quad (6)$$

IV. When $P \cup C$ is not a (k_1, k_2) -plex and the pivot vertex is in C (Lines 25–30). The cost in this case is

$$T(m, n) \leq c_2|V|^2 + T(m-1, n) + T(m-1, n-1).$$

In the worst case, no candidate vertex is reduced from $C - \{v_p\}$ in Line 28 so v_p remains the minimum degree vertex in the subsequent recursive call at Line 30 (within which $v_p \in P' = P \cup \{v_p\}$). Hence, by expanding $T(m-1, n)$ using Equation (5) we have:

$$T(m, n) \leq (p+2)c_2|V|^2 + \sum_{i=1}^p T(m-1-i, n-1) + T(m-1-d, n-d+p) + T(m-1, n-1). \quad (7)$$

Next, we compute a closed-form upper-bound of $T(m, n)$ which satisfies the Equations (4)–(7), and we can drop the second argument n from $T(m, n)$ since the base case expressions in Equations (4) and (6) are not a function of n . Then, the largest $T(m)$ is upper-bounded by Equations (4) and (7), i.e.

$$T(m) \leq \begin{cases} c_1 & \text{if } m = 0 \\ (p+2)c_2|V|^2 + \sum_{i=0}^p T(m-1-i) + T(m-1-d) & \text{otherwise} \end{cases} \quad (8)$$

Since $p \leq d-1$ and $p < k$, the worst-case is reached when $p = k-1$ and $d = k$. Then, Equation (8) can be rewritten as:

$$T(m) \leq \begin{cases} c_1 & \text{if } m = 0 \\ (k+1)c_2|V|^2 + \sum_{i=0}^k T(m-1-i) & \text{otherwise} \end{cases} \quad (9)$$

We show that $T(m) = \mathcal{O}(\alpha_k \gamma_k^m - \beta_k)$ where γ_k is largest real root of $x^{k+2} - 2x^{k+1} + 1 = 0$, $\beta_k = \frac{(k+1)c_2|V|^2}{k}$ and $\alpha_k = \frac{(2^k k + 1)\beta_k + 2^k c_1}{\gamma_k^{k+1}}$.

Lemma 1. Let $H(m) = \alpha_k \gamma_k^m - \beta_k$ where γ_k is largest real root of $x^{k+2} - 2x^{k+1} + 1 = 0$, $\beta_k = \frac{(k+1)c_2|V|^2}{k}$ and $\alpha_k = \frac{(2^k k + 1)\beta_k + 2^k c_1}{\gamma_k^{k+1}}$. If $T(m)$ satisfies Equation (9), then $T(m) \leq H(m)$ for $m \geq 0$.

Proof. We first show $T(m) \leq 2^{m-1}k\beta_k + 2^{m-1}c_1$ by induction on m .

Basis: for $m = 1$, $T(1) \leq (k+1)c_2|V|^2 + T(0) \leq k\beta_k + c_1$.

Induction: We assume that $T(i) \leq 2^{i-1}k\beta_k + 2^{i-1}c_1$ for all $i = 1, \dots, m-1$ and prove that it also holds for $i = m$. By Equation (9) we have:

$$\begin{aligned} T(m) &\leq (k+1)c_2|V|^2 + T(m-1) + \dots + T(0) \\ &= (k+1)c_2|V|^2 + c_1 + \sum_{i=1}^{m-1} T(i) \\ &\leq (k+1)c_2|V|^2 + c_1 + \sum_{i=1}^{m-1} (2^{i-1}k\beta_k + 2^{i-1}c_1) \\ &= k\beta_k(1 + \sum_{i=1}^{m-1} 2^{i-1}) + c_1(\sum_{i=1}^{m-1} 2^{i-1} + 1) \\ &= 2^{m-1}k\beta_k + 2^{m-1}c_1 \end{aligned} \quad (10)$$

Then, we prove that $T(m) \leq H(m)$ for all $m \geq 0$ again by induction on m .

Basis: for $m = 1$, $T(1) \leq (k+1)c_2|V|^2 + T(0) \leq k\beta_k + c_1$, and $H(1) = \alpha_k \gamma_k - \beta_k = \frac{(2^k k + 1)\beta_k + 2^k c_1}{\gamma_k^k} \geq k\beta_k + c_1$ (since $\gamma_k < 2$ and hence $\gamma_k^k < 2^k$).

Induction: We assume that $T(m) \leq H(m)$ for $m = 1, \dots, i$ and prove that it also holds for $m = i+1$. First, let us factorize $x^{k+1} - 2x^{k+1} + 1$ as $(x-1)(x^{k+1} - x^k - \dots - 1)$. Denote $F_k(x) = (x^{k+1} - x^k - \dots - 1)$. We have $F_k(1) < 0$ and $F_k(2) > 0$, indicating that $F_k(x) = 0$ has a root between $(1, 2)$. Combining the fact that γ_k is also the root of $F_k(x) = 0$, thus $\gamma_k^k + \dots + 1 = \gamma_k^{k+1}$. Multiply both side with γ_k^{i-k} , we

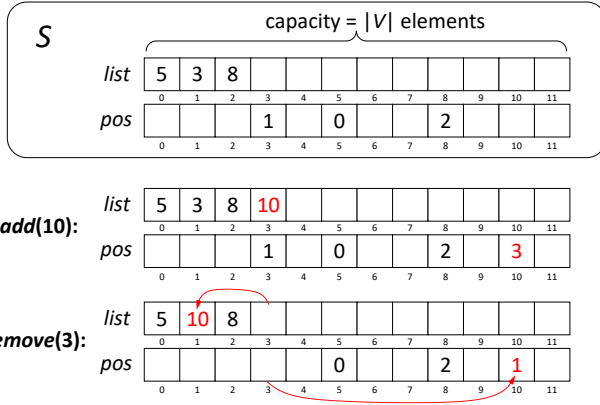


Fig. 10. Data Structure for Set Maintenance

have $\gamma_k^i + \dots + \gamma_k^{i-k} = \gamma_k^{i+1}$. Then the following inequalities hold.

$$\begin{aligned}
 T(i+1) &\leq T(i) + \dots + T(i-k) + (k+1)c_2|V|^2 \\
 &\leq \alpha_k(\gamma_k^i + \dots + \gamma_k^{i-k}) - (k+1)\beta_k + k\beta_k \\
 &\leq \alpha_k(\gamma_k^i + \dots + \gamma_k^{i-k}) - \beta_k \\
 &= \alpha_k\gamma_k^{i+1} - \beta_k \\
 &= H(i+1)
 \end{aligned} \tag{11}$$

□

Finally, as we call $BK2(\cdot)$ with $m = |V|$ and k is a constant, we get the worst-case running time of our algorithm as given in Theorem 8.

APPENDIX F

INCREMENTAL SET AND DEGREE MAINTENANCE

Instead of creating new sets P' , C' and X' as the inputs into each recursive call $BK2(G, k_1, k_2, q, P', C', X')$ (see Lines 17, 27 and 30), we propose to reuse the same containers for P , C and X throughout the recursive execution, by initializing their spaces at the very beginning, and populating them with proper elements for use in each recursion body.

In particular, we maintain three global containers for P , C and X , respectively, using the data structure shown in Figure 10. Specifically, two arrays with capacity $|V|$ are maintained for each set S : (1) $list[]$ which keeps the list of S 's elements for scanning and adding new elements; (2) $pos[]$ with which maps vertex ID back to its position in $list[]$, to facilitate element search and removal. Note that element addition, search and removal can all be done in $O(1)$ time. To illustrate using Figure 10, $S.add(10)$ appends 10 to $list[]$ at position 3, and records $pos[10] = 3$; while $S.remove(3)$ first finds the element position in $list[]$ as $pos[3] = 1$, and then erases it in $list[]$ by moving the last element 10 to $list[1]$, and moving 1 from $pos[3]$ to $pos[10]$. In this way, we can incrementally populate $\langle P, C, X \rangle$ for calling $BK2(\cdot)$ with minimal cost.

For example, for the else-branch in Lines 27–30, we can first conduct $C.remove(v_p)$ and $X.add(v_p)$ before calling $BK2(\cdot)$ at Line 27, and then recover X for use by Line 30 by conducting $X.remove(v_p)$. Line 28 (resp. Line 29) then removes from C

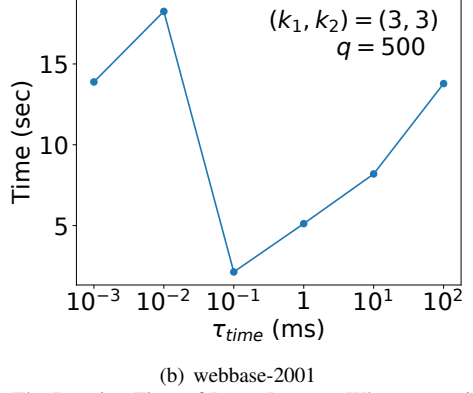
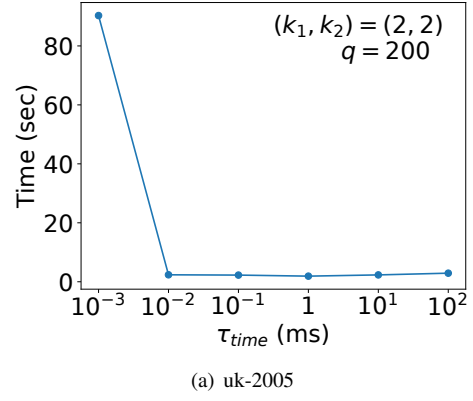


Fig. 11. The Running Time of Large Datasets When τ_{time} is Varied.

(resp. X) those elements v such that $P \cup \{v_p, v\}$ is a not (k_1, k_2) -plex, and let us denote the set of removed elements by C'' (resp. X''). Then, we can conduct $P.add(v_p)$ and call $BK(\cdot)$ with (P, C, X) in Line 30, after which we recover P, C and X for backtracking by conducting $P.remove(v_p)$, $C.add(C'')$ and $X.add(X'')$. Note that C'' and X'' are kept in global buffers whose spaces are initialized at the beginning, and reused throughout the recursive execution.

Finally, we also incrementally maintain (1) $d_{P \cup C}^+(\cdot)$ and $d_{P \cup C}^-(\cdot)$ which are needed in Lines 22, 28 and 29; and (2) $d_P^+(\cdot)$ and $d_P^-(\cdot)$ which are needed in Lines 8, 28 and 29, as we have explained at the end of Section III. For example, whenever we remove a vertex v from C , we need to decrement $d_{P \cup C}^-(u)$ for all $u \in N_{P \cup C}^+(v)$ and $d_{P \cup C}^+(u)$ for all $u \in N_{P \cup C}^-(v)$.

APPENDIX G

LOAD BALANCING

We also conducted experiments to study the impact of τ_{time} . Recall from Section VII that if a task runs for a period of more than τ_{time} , it will decompose itself so that the decomposed tasks can be processed in parallel. Figure 11 shows the impact of τ_{time} on the job running time, when using 32 threads and varying τ_{time} as 10^{-3} , 10^{-2} , 10^{-1} , 1, 10, 100 for two representative settings on our large graphs uk-2005 and webbase-2001. We can see that our default setting of $\tau_{time} = 0.1$ ms does work the best in general. Also, Figure 11(a) shows that if we set τ_{time} too small, too many tasks will be created so the task creation time dominates and the running time can be

TABLE V
THE RUNNING TIME OF MINING LARGE MAXIMAL (k_1, k_2) -PLEXES IN PARALLEL EXECUTION

Dataset	k_1	k_2	q	$\#(k_1, k_2)$ -plexes	Serial	2 Threads	4 Threads	8 Threads	16 Threads	32 Threads
bitcoin	2	5	10	144,00	0.06	0.04	0.02	0.01	0.01	0.02
			12	0	0.01	0.00	0.00	0.00	0.00	0.00
	3	5	10	24,261	8.35	4.78	2.46	1.29	0.66	0.36
			12	286	1.05	0.62	0.33	0.17	0.09	0.05
wiki-vote	3	5	10	474	1.85	1.08	0.55	0.29	0.15	0.08
			12	0	0.08	0.05	0.02	0.01	0.01	0.01
	4	5	10	30,222	349.32	197.94	102.45	52.57	26.57	16.44
			12	94	16.63	9.76	4.99	2.61	1.32	0.91
mathoverflow	2	2	10	523,633	28.55	18.48	9.48	4.93	2.50	1.34
			12	150,888	13.35	9.01	4.56	2.38	1.22	0.69
	2	3	10	1,762,917	63.16	47.81	24.41	12.71	6.45	3.52
			12	602,862	32.44	24.92	12.61	6.63	3.36	1.86
as-caida	2	3	10	23,314	7.99	0.34	0.19	0.10	0.08	0.15
			15	185	0.93	0.02	0.01	0.01	0.01	0.02
	3	4	15	17,303	14.87	0.63	0.33	0.18	0.10	0.10
			20	0	0.19	0.01	0.00	0.00	0.00	0.00
epinions	2	3	10	773,095	23.85	18.34	9.48	4.94	2.56	1.41
			12	252,511	10.33	8.06	4.18	2.23	1.19	0.80
	3	4	12	24,599,029	2,053.04	1,174.05	605.69	314.25	158.78	85.62
			15	2,382,084	381.99	229.27	118.32	61.53	31.62	18.13
email-euall	2	3	10	17,438	1.69	1.26	0.65	0.34	0.18	0.19
			12	1442	0.58	0.39	0.21	0.11	0.07	0.12
	3	4	12	196,878	80.58	46.44	23.49	12.18	6.14	3.38
			15	1351	9.19	5.20	2.68	1.40	0.72	0.44
amazon0505	3	5	10	4,696	0.27	0.08	0.04	0.02	0.02	0.01
			12	24	0.14	0.01	0.00	0.00	0.00	0.00
	5	3	10	1,532	0.20	0.04	0.02	0.01	0.01	0.00
			12	1	0.13	0.00	0.00	0.00	0.00	0.00
web-google	2	3	15	997	0.29	0.05	0.02	0.01	0.01	0.00
			20	57	0.21	0.01	0.01	0.00	0.00	0.00
	3	4	15	1,993	0.32	0.06	0.03	0.02	0.01	0.01
			20	61	0.22	0.02	0.01	0.00	0.00	0.00
soc-pokec	2	3	15	17,980	2.11	0.92	0.49	0.25	0.21	0.39
			20	17	0.95	0.03	0.01	0.01	0.01	0.02
	3	3	15	508,884	54.91	32.24	16.52	8.65	4.41	2.41
			20	648	1.66	0.49	0.25	0.13	0.07	0.06
wiki-talk	2	2	15	22,219	287.28	173.50	87.73	44.75	22.50	11.48
			20	0	69.70	38.84	19.89	10.27	5.17	2.67
	2	3	15	165,156	525.13	376.12	191.71	97.82	48.99	25.01
			20	0	129.02	77.30	39.23	20.22	10.16	5.21
arabic-2005	2	2	1000	106,895	3,906.70	1,897.35	1,017.31	527.40	269.04	140.58
			3000	3,500	111.60	60.20	29.56	15.10	8.69	4.64
uk-2005	2	2	200	117,255	301.09	162.70	84.11	43.34	21.82	11.38
			400	66,639	168.98	91.51	46.34	23.95	12.11	6.34
it-2004	2	2	1000	15,087	1,170.05	621.80	321.12	166.53	85.40	52.04
			3000	1,034	77.90	41.35	21.72	10.21	6.40	3.94
webbase-2001	2	2	300	327,882	646.41	320.84	164.46	90.02	53.56	30.75
			500	88,777	184.44	95.16	52.07	30	18.15	8.75
clue-web	2	2	10	1,215,977	26.05	14.15	7.66	3.82	1.75	1.69
			20	95,383	52.92	31.76	16.46	8.56	4.42	2.37

significantly increased, and Figure 11(b) shows that if we set τ_{time} too large, load balancing suffers so the running time can be many times longer.

APPENDIX H COMPLETE RESULTS IN PARALLEL EXECUTIONS

We implemented the parallel version of Ours based on the description in Section VII. Table V reports the (k_1, k_2) -plex enumeration time of parallel Ours on all our datasets in Table I when running with 2, 4, 8, 16 and 32 threads, respectively, where we use the default timeout threshold $\tau_{time} = 0.1$ ms is

adopted, which is tested to work consistently well on various datasets.

As Table V shows, our parallel algorithm is efficient and scales up well with the number of CPU cores on all the datasets.